# LAB 1 Елементи мови Python. Основні інструкції та структури даних. Робота з файловою системою

Нехай протягом навчання програмні проєкти з різних предметів зберігалися в папках ... . Є перелік папок `folders`, в яких зберігаються ці проєкти (задає виконавець, виходячи з вмісту своїх носіїв даних).
*Примітка*. До цього переліку мають входити не окремі папки з проєктами, а папки, що містять, наприклад, всі проєкти такої-то дисципліни такого-то семестру тощо. Інакше кажучи, не папка одного solution, а папка, в якій зберігаються solutions (якщо в термінології Visual Studio).

Щоб профільтрувати допоміжні файли середовищ, введемо "білий список" розширень ( `WHITE_LIST` ) та список розширень файлів з текстом програм(задається) ( `CODE_LIST` ).

З'ясувати

1. Скільки різних розширень зустрічається у папках з заданого списку (тут і далі: включаючи вкладені), скільки файлів з кожним розширенням. (Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)
   Відповідну інформацію вивести на екран охайною табличкою. На рядку виводиться розширення, кількість файлів та ознака (C,W,O відповідно для файлів з текстом програм (відповідно до `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.
   Наприклад,
   ```
   .txt     155    W
   .ipynb      73    W
   .cpp      45    C
   .exe       3    O
   ```

2. За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний обсяг у байтах, кількість їх рядків (враховувати тільки для файлів категорії C; білі та порожні рядки не рахувати), середній розмір файлу, середня кількість рядків.

Відповідну інформацію вивести на екран охайною табличкою в хронологічному порядку.

Наприклад,

```
2024  09   23   123456  256   5368   11
2024  10   23   123456  256   5368   11
2024  11  134   123456  256    921   11
```

3. Для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, знайти таку інформацію: рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків (тільки для розширень категорії C; білі та порожні рядки не рахувати; для інших розширень цю клітину таблиці залишаємо незаповненою).

   Цю інформацію записати в csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку додати рядок з заголовками: year, mon, ext, cnt, size, lines.

Здається:

1. Цей робочий зошит (заповнений, вилучати клітини, які були в ньому з самого початку, не дозволяється) та pdf-звіт.
   Перед здачею необхідно зробити Kernel -> Restart Kernel and Run All Cells... Для отримання pdf-звіта блокнот можна просто роздрукувати.
   **Увага!** Повідомлень про помилки виконання бути не повинно.
2. Файли зі скриптами для вирішення трьох задач (див. вище).
3. Файл `lab1_1.csv`

Оцінювання (максимум 15 балів):

Обов'язково має бути повністю та правильно виконаний (без повідомлень про помилки) та заповнений робочий зошит (блокнот), має бути pdf-звіт та `lab1_1.csv`, що відповідає виконаному блокноту.

За відсутності скриптів оцінка не може перевищувати 12 балів.

Правильність виконання оцінюється з точки зору дотримання вимог та отриманого кінцевого результату.

Якщо було вибрано папки майже без коду, то оцінка не може перевищувати 6

балів.

За неохайне виведення знімається 3 бали.

In [1]:
```
CODE_LIST = {'.cpp', '.hpp', '.c', '.h', '.py', '.pyw', '.cs', '.js', '.java
WHITE_LIST = CODE_LIST | {'.html', '.json', '.txt', '.ipynb'}
```

In [2]:
```
import sys
print(sys.platform, sys.version, sep='\n')
```

```
darwin
3.14.2 (v3.14.2:df793163d58, Dec  5 2025, 12:18:06) [Clang 16.0.0 (clang-160
0.0.26.6)]
```

Інструкції імпортування зазвичай записують на початку. Далі вони будуть додатково продубльовані по місцю використання.

In [3]:
```
import datetime
from pathlib import Path
from datetime import datetime as dt
import csv
```

# Вихідні відомості

In [4]:
```
language = 'python3'
```

Бібліотека `datetime` містить стандартні засоби для зображення дат, зокрема клас datetime.date

In [5]:
```
import datetime
year = datetime.date.today().year
```

In [6]:
```
year
```

Out[6]:
```
2026
```

In [7]:
```
f'Ця робота використовує мову програмування {language} і була виконана у {ye
```

Out[7]:
```
'Ця робота використовує мову програмування python3 і була виконана у 2026 р
оці.'
```

In [8]:
```
author = "Роман Бирак"
group = "К-26"
lab_instructor = "Ляшко А.В."
```

Запишіть f-рядок, значенням якого є текст (розташовується на двох рядках)
`Виконавець: ...., група ...`
`Викладач:`
Значення виконавця, його групи та викладача мають підставлятися зі змінних `author`, `group` та `lab_instructor`.

▼ Підказка.
Доцільно використати рядок у трикратних лапках.

In [9]:
```
info = f"""Виконавець: {author}, група {group}
Викладач: {lab_instructor}"""
print(info)
```

```
Виконавець: Роман Бирак, група К-26
Викладач: Ляшко А.В.
```

# Готуємось "гуляти" по папках (підготовка-дослідження)

Нам знадобиться клас `Path` з бібліотеки `pathlib`. Документація.

In [10]:
```
from pathlib import Path
```

Дізнаємось поточний робочий каталог. (**c**urrent **w**ork **d**ir)

In [11]:
```
Path().cwd()
```

Out[11]:
```
PosixPath('/Users/yaroslavbyrak/PycharmProjects/python-course-university/lab1')
```

In [12]:
```
str(Path().cwd())
```

Out[12]:
```
'/Users/yaroslavbyrak/PycharmProjects/python-course-university/lab1'
```

Шлях до якої-небудь папки з помірною кількістю файлів з різними розширеннями та складеними папками зробити значенням змінної `root`.
У кожного має бути свій шлях.

Наприклад, можна взяти папку з проєктом мови C++.

In [13]:
```
root = Path('/Users/yaroslavbyrak/CLionProjects/untitled116')
root
```

Out[13]:
```
PosixPath('/Users/yaroslavbyrak/CLionProjects/untitled116')
```

## Обійти папку, вивести всі наявні в ній файли

Цікавить шлях, тип файлу (каталог чи звичайний файл), для звичайного файлу цікавить розширення та розмір у байтах.

Сімлінки та інше будемо пропускати.

▶ Підказка 1

In [14]:
```
help(Path.glob)
```

```
Help on function glob in module pathlib:

glob(self, pattern, *, case_sensitive=None, recurse_symlinks=False)
    Iterate over this subtree and yield all existing files (of any
    kind, including directories) matching the given relative pattern.
```

▼ Підказка 2

Для рекурсивного обходу каталогу можна використати шаблон `**/*`, за яким будуть обходитись усі елементи, крім самої папки, що обходиться ( . ).

▼ Підказка 3

Якщо `s` задає послідовність або те, по чому можна ітерувати, то обхід можна реалізувати за допомогою циклу `for`

`for el in s:`

`do something with s`

Обійти всі наявні в `root` файли та вивести їх імена

In [15]:
```
for el in root.glob("**/*"):
    print(el)# заповнити
```

```
/Users/yaroslavbyrak/CLionProjects/untitled116/results_O0_final.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/main_final
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug
/Users/yaroslavbyrak/CLionProjects/untitled116/FINAL_REPORT.md
/Users/yaroslavbyrak/CLionProjects/untitled116/main_O0_final
/Users/yaroslavbyrak/CLionProjects/untitled116/main_O3
/Users/yaroslavbyrak/CLionProjects/untitled116/CMakeLists.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/.DS_Store
/Users/yaroslavbyrak/CLionProjects/untitled116/BUILD_INSTRUCTIONS.md
/Users/yaroslavbyrak/CLionProjects/untitled116/main_O0
/Users/yaroslavbyrak/CLionProjects/untitled116/ANALYSIS_REPORT.md
/Users/yaroslavbyrak/CLionProjects/untitled116/results_O3_final.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/results_O0.txt
```

```
/Users/yaroslavbyrak/CLionProjects/untitled116/FINAL_BUILD_INSTRUCTIONS.md
/Users/yaroslavbyrak/CLionProjects/untitled116/results_O3.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/main
/Users/yaroslavbyrak/CLionProjects/untitled116/main.cpp
/Users/yaroslavbyrak/CLionProjects/untitled116/output.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.ninja_deps
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/cmake_insta
ll.cmake
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/build.ninja
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/Testing
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeCache.
txt
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/untitled116
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.ninja_log
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea/editor.xml
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea/vcs.xml
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea/.gitignore
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea/workspace.xml
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea/modules.xml
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea/untitled116.iml
/Users/yaroslavbyrak/CLionProjects/untitled116/.idea/misc.xml
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
untitled116.dir
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
clion-Debug-log.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
cmake.check_cache
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
CMakeConfigureLog.yaml
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
rules.ninja
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
CMakeScratch
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
pkgRedirects
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
TargetDirectories.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
clion-environment.txt
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/Testing/Tem
porary
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
```

```
v1/reply
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/query
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/reply/toolchains-v1-21caedbeaa7fbcc9ce56.json
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/reply/directory-.-Debug-f5ebdc15457944623624.json
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/reply/cache-v2-0e458666cafe4baafa66.json
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/reply/cmakeFiles-v1-7ac2e0a6726ef3966eb1.json
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/reply/index-2025-10-12T11-01-35-0479.json
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/reply/codemodel-v2-534e9e281148afff7827.json
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/reply/target-untitled116-Debug-bbae802e0eb3a36057d4.json
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/query/toolchains-v1
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/query/codemodel-v2
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/query/cmakeFiles-v1
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/.cmake/api/
v1/query/cache-v2
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/Testing/Tem
porary/LastTest.log
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CMakeDetermineCompilerABI_CXX.bin
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CMakeDetermineCompilerABI_C.bin
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CompilerIdC
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CompilerIdCXX
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CMakeCXXCompiler.cmake
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CMakeCCompiler.cmake
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CMakeSystem.cmake
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CompilerIdC/CMakeCCompilerId.c
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CompilerIdC/a.out
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CompilerIdC/tmp
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CompilerIdCXX/a.out
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
```

```
3.29.6/CompilerIdCXX/CMakeCXXCompilerId.cpp
/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debug/CMakeFiles/
3.29.6/CompilerIdCXX/tmp
```

Є бажання виводити їх шляхи відносно заданої папки.

Тобто необхідно побудувати шлях, відносно шляху `root`. Допоможе метод `relative_to`.

In [16]:
```
help(Path.relative_to)
```

```
Help on function relative_to in module pathlib:

relative_to(self, other, *, walk_up=False)
    Return the relative path to another path identified by the passed
    arguments.  If the operation is not possible (because this is not
    related to the other path), raise ValueError.

    The *walk_up* parameter controls whether `..` may be used to resolve
    the path.
```

*Примітка*. Насправді це метод класу `PurePath`, який є базовим для `Path`.

Після завершення циклу змінна `el`, яка позначала поточний елемент обходу, продовжує існувати. Проведемо з нею кілька експериментів.

Виведемо її значення.

In [17]:
```
el
```

Out[17]:
```
PosixPath('/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debu
g/CMakeFiles/3.29.6/CompilerIdCXX/tmp')
```

У наступній клітині побудуємо та виведемо відповідний відносний шлях.

▼ Підказка

`el.relative_to(root)`

In [18]:
```
el.relative_to(root)
```

Out[18]:
```
PosixPath('cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/tmp')
```

Явно перетворимо відносний шлях на рядок (значення типу `str`).

▼ Підказка

Щоб об'єкт `obj` перетворити на об'єкт типу `T`, можна використати конструктор

типу `T`, а саме `T(obj)`.

▼ Відповідь

`str(el.relative_to(root))`

In [19]:
```
str(el.relative_to(root))
```

Out[19]:
```
'cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/tmp'
```

Тепер у нашому обході будемо виводити шляхи, відносно шляху `root`. У наступну клітину слід записати модифікацію обходу, який здійснювався раніше.

*Примітка*. Якщо ми збираємось відносний шлях тільки виводити, то перед виведенням перетворювати його явно на рядок потреби немає. Це чудово зробить сама функція `print`.

▼ Підказка

`for el in` *`вираз, що задає обхід`*`:`
`    print(відносний шлях)`

In [20]:
```
for el in root.glob("**/*"):
    print(el.relative_to(root))
```

```
results_O0_final.txt
main_final
cmake-build-debug
FINAL_REPORT.md
main_O0_final
main_O3
CMakeLists.txt
.DS_Store
BUILD_INSTRUCTIONS.md
main_O0
ANALYSIS_REPORT.md
results_O3_final.txt
results_O0.txt
FINAL_BUILD_INSTRUCTIONS.md
results_O3.txt
main
main.cpp
output.txt
.idea
cmake-build-debug/CMakeFiles
cmake-build-debug/.ninja_deps
cmake-build-debug/cmake_install.cmake
cmake-build-debug/build.ninja
```

```
cmake-build-debug/Testing
cmake-build-debug/CMakeCache.txt
cmake-build-debug/untitled116
cmake-build-debug/.ninja_log
cmake-build-debug/.cmake
.idea/editor.xml
.idea/vcs.xml
.idea/.gitignore
.idea/workspace.xml
.idea/modules.xml
.idea/untitled116.iml
.idea/misc.xml
cmake-build-debug/CMakeFiles/untitled116.dir
cmake-build-debug/CMakeFiles/clion-Debug-log.txt
cmake-build-debug/CMakeFiles/cmake.check_cache
cmake-build-debug/CMakeFiles/CMakeConfigureLog.yaml
cmake-build-debug/CMakeFiles/rules.ninja
cmake-build-debug/CMakeFiles/CMakeScratch
cmake-build-debug/CMakeFiles/3.29.6
cmake-build-debug/CMakeFiles/pkgRedirects
cmake-build-debug/CMakeFiles/TargetDirectories.txt
cmake-build-debug/CMakeFiles/clion-environment.txt
cmake-build-debug/Testing/Temporary
cmake-build-debug/.cmake/api
cmake-build-debug/.cmake/api/v1
cmake-build-debug/.cmake/api/v1/reply
cmake-build-debug/.cmake/api/v1/query
cmake-build-debug/.cmake/api/v1/reply/toolchains-v1-21caedbeaa7fbcc9ce56.jso
n
cmake-build-debug/.cmake/api/v1/reply/directory-.-Debug-f5ebdc1545794462362
4.json
cmake-build-debug/.cmake/api/v1/reply/cache-v2-0e458666cafe4baafa66.json
cmake-build-debug/.cmake/api/v1/reply/cmakeFiles-v1-7ac2e0a6726ef3966eb1.jso
n
cmake-build-debug/.cmake/api/v1/reply/index-2025-10-12T11-01-35-0479.json
cmake-build-debug/.cmake/api/v1/reply/codemodel-v2-534e9e281148afff7827.json
cmake-build-debug/.cmake/api/v1/reply/target-untitled116-Debug-bbae802e0eb3a
36057d4.json
cmake-build-debug/.cmake/api/v1/query/toolchains-v1
cmake-build-debug/.cmake/api/v1/query/codemodel-v2
cmake-build-debug/.cmake/api/v1/query/cmakeFiles-v1
cmake-build-debug/.cmake/api/v1/query/cache-v2
cmake-build-debug/Testing/Temporary/LastTest.log
cmake-build-debug/CMakeFiles/3.29.6/CMakeDetermineCompilerABI_CXX.bin
cmake-build-debug/CMakeFiles/3.29.6/CMakeDetermineCompilerABI_C.bin
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX
cmake-build-debug/CMakeFiles/3.29.6/CMakeCXXCompiler.cmake
cmake-build-debug/CMakeFiles/3.29.6/CMakeCCompiler.cmake
cmake-build-debug/CMakeFiles/3.29.6/CMakeSystem.cmake
```

```
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC/CMakeCCompilerId.c
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC/a.out
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC/tmp
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/a.out
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/CMakeCXXCompilerId.cpp
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/tmp
```

## Складові шляху

Інформацію про частини шляху можна знайти тут.

Альтернатива: використати довідку.

Знайдемо всі атрибути, доступні для класу `Path`. (У Python під атрибутами розуміють і класичні атрибути (описують дані об'єкта), і методи (це наслідок технічної реалізації методів).)

Функція `dir` поверне список.

In [21]:
```
dir(Path)
```

Out[21]:
```
['__bytes__',
 '__class__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__firstlineno__',
 '__format__',
 '__fspath__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__rtruediv__',
 '__setattr__',
 '__sizeof__',
```

```
    '__slots__',
    '__static_attributes__',
    '__str__',
    '__subclasshook__',
    '__truediv__',
    '_copy_from',
    '_copy_from_file',
    '_copy_from_symlink',
    '_delete',
    '_drv',
    '_filter_trailing_slash',
    '_format_parsed_parts',
    '_from_dir_entry',
    '_from_parsed_parts',
    '_from_parsed_string',
    '_hash',
    '_info',
    '_parse_path',
    '_parse_pattern',
    '_parts_normcase',
    '_parts_normcase_cached',
    '_raw_path',
    '_raw_paths',
    '_remove_leading_dot',
    '_remove_trailing_slash',
    '_root',
    '_str',
    '_str_normcase',
    '_str_normcase_cached',
    '_tail',
    '_tail_cached',
    'absolute',
    'anchor',
    'as_posix',
    'as_uri',
    'chmod',
    'copy',
    'copy_into',
    'cwd',
    'drive',
    'exists',
    'expanduser',
    'from_uri',
    'full_match',
    'glob',
    'group',
    'hardlink_to',
    'home',
    'info',
    'is_absolute',
```

```
    'is_block_device',
    'is_char_device',
    'is_dir',
    'is_fifo',
    'is_file',
    'is_junction',
    'is_mount',
    'is_relative_to',
    'is_reserved',
    'is_socket',
    'is_symlink',
    'iterdir',
    'joinpath',
    'lchmod',
    'lstat',
    'match',
    'mkdir',
    'move',
    'move_into',
    'name',
    'open',
    'owner',
    'parent',
    'parents',
    'parser',
    'parts',
    'read_bytes',
    'read_text',
    'readlink',
    'relative_to',
    'rename',
    'replace',
    'resolve',
    'rglob',
    'rmdir',
    'root',
    'samefile',
    'stat',
    'stem',
    'suffix',
    'suffixes',
    'symlink_to',
    'touch',
    'unlink',
    'walk',
    'with_name',
    'with_segments',
    'with_stem',
    'with_suffix',
    'write_bytes',
```

```
 'write_text']
```

Серед них ті, що починаються з двох підкреслень, є службовими. Ті, що починаються з одного підкреслення, — допоміжними (не є відкритими). Виведемо всі, крім них.

In [22]:
```
cnt = 0
for a in dir(Path):
    if not a.startswith('_'):
        print(a, end=', ')
        cnt += 1
        if (cnt > 5):
            cnt = 0
            print()
```

```
absolute, anchor, as_posix, as_uri, chmod, copy,
copy_into, cwd, drive, exists, expanduser, from_uri,
full_match, glob, group, hardlink_to, home, info,
is_absolute, is_block_device, is_char_device, is_dir, is_fifo, is_file,
is_junction, is_mount, is_relative_to, is_reserved, is_socket, is_symlink,
iterdir, joinpath, lchmod, lstat, match, mkdir,
move, move_into, name, open, owner, parent,
parents, parser, parts, read_bytes, read_text, readlink,
relative_to, rename, replace, resolve, rglob, rmdir,
root, samefile, stat, stem, suffix, suffixes,
symlink_to, touch, unlink, walk, with_name, with_segments,
with_stem, with_suffix, write_bytes, write_text,
```

Повертаємось до нашого примірника шляху.

In [23]:
```
el
```

Out[23]:
```
PosixPath('/Users/yaroslavbyrak/CLionProjects/untitled116/cmake-build-debu
g/CMakeFiles/3.29.6/CompilerIdCXX/tmp')
```

Визначимо, чи задає цей шлях папку, файл, яке в нього ім'я, розширення, який його розмір.

▼ Підказка

Використати атрибути `is_dir`, `is_file`, `name`, `suffix`, `stat().st_size`

Записати вираз, що перевіряє, чи задає шлях `el` папку.

In [24]:
```
el.is_dir()# заповнити
```

Out[24]:
```
True
```

Записати вираз, що перевіряє, чи задає шлях `el` файл.

In [25]:
```
el.is_file()# заповнити
```

Out[25]:
```
False
```

Записати вираз, що визначає ім'я файлу `el`.

In [26]:
```
el.name# заповнити
```

Out[26]:
```
'tmp'
```

Записати вираз, що визначає розширення файлу `el`.

In [27]:
```
el.suffix# заповнити
```

Out[27]:
```
''
```

Записати вираз, що визначає розмір файлу `el`.

In [28]:
```
el.stat().st_size# заповнити
```

Out[28]:
```
64
```

Для розв'язання загальної задачі нам буде необхідно вміти визначати рік та місяць створення файла.

Не тільки розмір файлу, але й інша інформація про файл може бути знайдена за допомогою метода `stat` (документація)

Нас цікавить дата створення файла. Тут виникає невелика залежність від ОС. (документація)

Знайдемо час створення файлу (у секундах) і запам'ятаємо в змінній `ctime`.

In [29]:
```
if sys.platform.startswith('win'):
    ctime = el.stat().st_birthtime
else:
    ctime = el.stat().st_ctime
ctime
```

Out[29]:
```
1760266894.0929961
```

Тепер слід перетворити на звичайну дату, щоб з'ясувати рік та місяць.

Використаємо клас `datetime`, точніше його метод `fromtimestamp`.

(документація)

Нижче клас `datetime` імпортований з одноіменної бібліотеки `datetime` під псевдонімом `dt`.

In [30]:
```
from datetime import datetime as dt
```

In [31]:
```
cdate = dt.fromtimestamp(ctime)
cdate
```

Out[31]:
```
datetime.datetime(2025, 10, 12, 14, 1, 34, 92996)
```

In [32]:
```
cdate.year, cdate.month
```

Out[32]:
```
(2025, 10)
```

Напишіть **кросплатформену** функцію `created_at`, що за шляхом до файлу визначає рік та місяць його створення. Функція має отримувати шлях до файлу та повертати пару цілих чисел.

Приклад функції, що повертає два значення.
`def f(arg):`
`return arg, arg+arg`

**Питання для обговорення**. В який момент краще розгалужити обчислення дати створення?

▼ Відповідь

```
def _created_at_Win(...): ...

def _created_at_Linux(...): ...

if sys.platform.startswith('win'):
created_at = _created_at_Win
else:
created_at = _created_at_Linux
```

In [33]:
```
def _created_at_Linux(path):     # треба записати власні реалізації цих двох
    source_path = Path(path)
    ctime = source_path.stat().st_ctime
```

```
    cdate = dt.fromtimestamp(ctime)
    return cdate.year, cdate.month

def _created_at_Windows(path):
    source_path = Path(path)
    ctime = source_path.stat().st_birthtime
    cdate = dt.fromtimestamp(ctime)
    return cdate.year, cdate.month

if sys.platform.startswith('win'):
    created_at = _created_at_Windows
else:
    created_at = _created_at_Linux
```

Перевіримо працездатність функції.

In [34]:
```
created_at(el)
```

Out[34]:
```
(2025, 10)
```

In [35]:
```
y, m = created_at(el)
print(y, m)
```

```
2025 10
```

Повертаємось до задачі обходу та виведення

Тепер для шляху `el` виведемо:
відповідний йому відносний шлях, далі на тому ж рядку `file` або `dir`
для файлу на новому рядку з абзацного відступу виведемо його розширення в 10 символів та розмір в 10 символів

In [36]:
```
s = '.txt'
size = 12345
print(f'\t {s:10} {size:10}')
```

```
        .txt            12345
```

Записати код, що виводить інформацію про файл як у прикладі нижче

In [37]:
```
def get_path_type(path) -> str:
    if path.is_file():
        return "file"
    return "dir"

def print_info(path, path_root):
    result = f"{path.relative_to(path_root)} --- {get_path_type(path)}"
    if path.is_file():
```

```
        result += f"\n\t {path.suffix:10} {path.stat().st_size:10}"
    print(result)

print_info(el, root)
```

```
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/tmp --- dir
```

*Приклад формату виведення*: `build/.cmake/api/v1/query/client-vscode/query.jsonfile --- file .json 144`

Зберемо обхід, за якого будемо виводити інформацію тільки про папки та звичайні файли.

У базовий обхід додаємо виведення інформації про файли.

In [38]:
```
for el in root.glob("**/*"):
    if el.is_dir() or el.is_file():
        print_info(el, root)
```

```
results_O0_final.txt --- file
         .txt             5193
main_final --- file
                         28848
cmake-build-debug --- dir
FINAL_REPORT.md --- file
         .md              4593
main_O0_final --- file
                        141848
main_O3 --- file
                         40280
CMakeLists.txt --- file
         .txt              359
.DS_Store --- file
                          6148
BUILD_INSTRUCTIONS.md --- file
         .md              1183
main_O0 --- file
                        252928
ANALYSIS_REPORT.md --- file
         .md              4111
results_O3_final.txt --- file
         .txt             5181
results_O0.txt --- file
         .txt             5855
FINAL_BUILD_INSTRUCTIONS.md --- file
         .md              1758
results_O3.txt --- file
         .txt             5852
```

```
main --- file
                        36168
main.cpp --- file
         .cpp            5161
output.txt --- file
         .txt               0
.idea --- dir
cmake-build-debug/CMakeFiles --- dir
cmake-build-debug/.ninja_deps --- file
                       115424
cmake-build-debug/cmake_install.cmake --- file
         .cmake          1546
cmake-build-debug/build.ninja --- file
         .ninja         27942
cmake-build-debug/Testing --- dir
cmake-build-debug/CMakeCache.txt --- file
         .txt           15078
cmake-build-debug/untitled116 --- file
                       364456
cmake-build-debug/.ninja_log --- file
                          208
cmake-build-debug/.cmake --- dir
.idea/editor.xml --- file
         .xml           83260
.idea/vcs.xml --- file
         .xml             137
.idea/.gitignore --- file
                          176
.idea/workspace.xml --- file
         .xml            5270
.idea/modules.xml --- file
         .xml             274
.idea/untitled116.iml --- file
         .iml              97
.idea/misc.xml --- file
         .xml             892
cmake-build-debug/CMakeFiles/untitled116.dir --- dir
cmake-build-debug/CMakeFiles/clion-Debug-log.txt --- file
         .txt            1554
cmake-build-debug/CMakeFiles/cmake.check_cache --- file
         .check_cache       85
cmake-build-debug/CMakeFiles/CMakeConfigureLog.yaml --- file
         .yaml          69566
cmake-build-debug/CMakeFiles/rules.ninja --- file
         .ninja          3534
cmake-build-debug/CMakeFiles/CMakeScratch --- dir
cmake-build-debug/CMakeFiles/3.29.6 --- dir
cmake-build-debug/CMakeFiles/pkgRedirects --- dir
cmake-build-debug/CMakeFiles/TargetDirectories.txt --- file
         .txt             277
```

```
cmake-build-debug/CMakeFiles/clion-environment.txt --- file
         .txt              285
cmake-build-debug/Testing/Temporary --- dir
cmake-build-debug/.cmake/api --- dir
cmake-build-debug/.cmake/api/v1 --- dir
cmake-build-debug/.cmake/api/v1/reply --- dir
cmake-build-debug/.cmake/api/v1/query --- dir
cmake-build-debug/.cmake/api/v1/reply/toolchains-v1-21caedbeaa7fbcc9ce56.jso
n --- file
         .json            2967
cmake-build-debug/.cmake/api/v1/reply/directory-.-Debug-f5ebdc1545794462362
4.json --- file
         .json             154
cmake-build-debug/.cmake/api/v1/reply/cache-v2-0e458666cafe4baafa66.json ---
file
         .json           23682
cmake-build-debug/.cmake/api/v1/reply/cmakeFiles-v1-7ac2e0a6726ef3966eb1.jso
n --- file
         .json           28881
cmake-build-debug/.cmake/api/v1/reply/index-2025-10-12T11-01-35-0479.json --
- file
         .json            1896
cmake-build-debug/.cmake/api/v1/reply/codemodel-v2-534e9e281148afff7827.json
--- file
         .json             988
cmake-build-debug/.cmake/api/v1/reply/target-untitled116-Debug-bbae802e0eb3a
36057d4.json --- file
         .json            1929
cmake-build-debug/.cmake/api/v1/query/toolchains-v1 --- file
                             0
cmake-build-debug/.cmake/api/v1/query/codemodel-v2 --- file
                             0
cmake-build-debug/.cmake/api/v1/query/cmakeFiles-v1 --- file
                             0
cmake-build-debug/.cmake/api/v1/query/cache-v2 --- file
                             0
cmake-build-debug/Testing/Temporary/LastTest.log --- file
         .log              123
cmake-build-debug/CMakeFiles/3.29.6/CMakeDetermineCompilerABI_CXX.bin --- fi
le
         .bin            16984
cmake-build-debug/CMakeFiles/3.29.6/CMakeDetermineCompilerABI_C.bin --- file
         .bin            17000
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC --- dir
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX --- dir
cmake-build-debug/CMakeFiles/3.29.6/CMakeCXXCompiler.cmake --- file
         .cmake           6753
cmake-build-debug/CMakeFiles/3.29.6/CMakeCCompiler.cmake --- file
         .cmake           3586
cmake-build-debug/CMakeFiles/3.29.6/CMakeSystem.cmake --- file
```

```
        .cmake            360
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC/CMakeCCompilerId.c --- file
        .c              27496
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC/a.out --- file
        .out            33736
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdC/tmp --- dir
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/a.out --- file
        .out            33736
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/CMakeCXXCompilerId.cpp ---
file
        .cpp            27064
cmake-build-debug/CMakeFiles/3.29.6/CompilerIdCXX/tmp --- dir
```

Визначимо, файли з якими розширеннями присутні в папці, що обходиться.

▼ Підказка 1 (про алгоритм).

Створимо порожню множину. У процесі обходу папки будемо додавати в неї знайдені розширення. Наприкінці виведемо знайдені розширення в алфавітному порядку.

▼ Підказка 2 (про деталі реалізації).

- Типом множин є `set` (документація).
- Щоб додати елемент до множини, можна використати метод `add` (документація).
- Відсортувати допоможе функція `sorted` (документація). Зверніть увагу на те, як задати сортування у спадному порядку.

Значенням змінної `suffixes` зробіть множину наявних розширень файлів.

In [39]:
```
suffixes = set()
for el in root.glob("**/*"):
    if el.is_file() and el.suffix is not None and el.suffix != "":
        suffixes.add(el.suffix)
```

In [40]:
```
suffixes
```

Out[40]:
```
{'.bin',
 '.c',
 '.check_cache',
 '.cmake',
 '.cpp',
 '.iml',
 '.json',
 '.log',
 '.md',
 '.ninja',
 '.out',
 '.txt',
 '.xml',
 '.yaml'}
```

Перетворіть її на відсортований список `suffixes_sorted`.

In [41]:
```
suffixes_sorted = sorted(suffixes) # заповнити
```

Виведіть знайдені розширення у відсортованому порядку.

In [42]:
```
for el in suffixes_sorted:
    print(el, end=', ')
print()
```

```
.bin, .c, .check_cache, .cmake, .cpp, .iml, .json, .log, .md, .ninja, .out,
.txt, .xml, .yaml,
```

Скільки різних розширень було знайдено? (Виведіть це значення. Вручну нічого не рахуємо!)

▼ Підказка
Функція `len` допоможе знайти кількість елементів довільного контейнера.

In [43]:
```
len(suffixes_sorted)
```

Out[43]: 14

Знайдемо скільки файлів з певним розширенням наявно у досліджуваній папці.

Для кожного розширення визначимо, скільки файлів з цим розширенням присутні в папці, що обходиться.

▼ Підказка 1 (про алгоритм).
Створимо словник, ключами в якому будуть розширення файлів. Значеннями словника будуть лічильники кількості файлів з відповідним розширенням. У процесі

обходу папки будемо збільшувати лічильники.

▼ Підказка 2 (про деталі реалізації).

- Типом словників є `dict` (документація).
- Наявність ключа в словнику можна перевірити за допомогою оператора `in`. Або у нагоді може стати метод `setdefault`.

Значенням змінної `d` зробіть словник, що знає кількість файлів для кожного наявного розширення.

In [44]:
```
d ={}# заповнити
for el in root.glob("**/*"):
    if el.is_file() and el.suffix != None and el.suffix != "":
        if el.suffix not in d:
            d[el.suffix] = 1
        else:
            d[el.suffix] += 1
```

In [45]:
```
d
```

Out[45]:
```
{'.txt': 10,
 '.md': 4,
 '.cpp': 2,
 '.cmake': 4,
 '.ninja': 2,
 '.xml': 5,
 '.iml': 1,
 '.check_cache': 1,
 '.yaml': 1,
 '.json': 7,
 '.log': 1,
 '.bin': 2,
 '.c': 1,
 '.out': 2}
```

In [46]:
```
suffixes_cnts = list(d.items())
for el in suffixes_cnts:
    print(el, end=', ')
print()
```

```
('.txt', 10), ('.md', 4), ('.cpp', 2), ('.cmake', 4), ('.ninja', 2), ('.xm
l', 5), ('.iml', 1), ('.check_cache', 1), ('.yaml', 1), ('.json', 7), ('.lo
g', 1), ('.bin', 2), ('.c', 1), ('.out', 2),
```

Знайдемо найбільш популярні розширення.

▼ Підказки

1. Серед вбудованих функцій Python зверніть увагу на `max` (документація).
2. Анонімні функції теж будуть у нагоді.

Є список пар.

In [47]:
```
lst = [(1, 2), (1, 5), (8, 2), (2, 4)]
lst
```

Out[47]:
```
[(1, 2), (1, 5), (8, 2), (2, 4)]
```

Відсортуємо пари лексикографічно.

In [48]:
```
lst.sort()
lst
```

Out[48]:
```
[(1, 2), (1, 5), (2, 4), (8, 2)]
```

Відсортуємо пари за ключем. В якості ключа візьмемо обернену пару.

In [49]:
```
lst.sort(key = lambda el: (el[1], el[0]))
lst
```

Out[49]:
```
[(1, 2), (8, 2), (2, 4), (1, 5)]
```

Знайти найбільшу кількість файлів, що мають однакове розширення. Присвоїти значення змінній `max_count`.

▼ Підказка
Ключем має бути другий елемент пари.

In [50]:
```
max_count = max(suffixes_cnts, key=lambda el: el[1])
max_count
```

Out[50]:
```
('.txt', 10)
```

In [51]:
```
max_count = max_count[1]
max_count
```

Out[51]:
```
10
```

Вивести всі розширення, що відповідають цій найбільшій кількості входжень.

▼ Підказка

Як варіант: обійти пари `.items()`, що зберігаються в словнику, і вивести потрібні розширення.

In [52]:
```
for suffix, file_count in d.items():
    if file_count == max_count:
        print(suffix)
```

```
.txt
```

**Класифікація розширень**

Як за розширенням визначити, чи потрапляє воно в задані переліки?

▼ Підказка
Використайте оператор `in`

Запишіть логічний вираз, що задає ознаку того, що розширення .txt наявне у білому списку.

▼ Відповідь

`'.txt' in WHITE_LIST`

In [53]:
```
".txt" in WHITE_LIST
```

Out[53]:
```
True
```

Напишіть функцію `kind`, що за розширенням повертає рядок з символом W,C,O (див. умову)

In [54]:
```
def kind(extension):# записати
    if extension in CODE_LIST:
        return "C"
    if extension in WHITE_LIST:
        return "W"
    return "O"
```

Модифікуйте словник так, щоб для кожного розширення він зберігав:

1. кількість файлів
2. їх сумарний розмір
3. їх сумарну кількість рядків тексту (тільки для файлів з текстом програм, для інших — $0$)

Значеннями словника можуть бути списки з трьох елементів (кількість файлів,

сумарний розмір, сумарна кількість рядків тексту)

In [55]:
```
sample = {'.cpp':[23, 123456, 56]}   # приклад
```

При перегляді файлів слід оновлювати поля значення. Наприклад `sample['.cpp'][0]+=1` збільшує кількість файлів з розширенням на 1 `sample['.cpp'][1]+=1234` додає розмір файлу (1234) до сумарного розміру

Якщо треба рахувати інформацію не тільки в розрізі розширень, а ще й місяців, то в якості ключів можна використати трійки: рік, місяць, розширення.

In [56]:
```
sample = {(2024, 9, '.cpp'):[23, 123456, 56]}   # приклад
sample
```

Out[56]:
```
{(2024, 9, '.cpp'): [23, 123456, 56]}
```

In [57]:
```
sample[2024, 9, '.cpp']
```

Out[57]:
```
[23, 123456, 56]
```

Кількість рядків файлу

Створити шлях до якого-небудь власного файлу з текстом програми. (Підставити власний шлях.)

In [58]:
```
sample = root.joinpath('main.cpp')
```

In [59]:
```
sample.stat().st_size
```

Out[59]:
```
5161
```

Знайдемо кількість рядків файлу

In [60]:
```
if sample.stat().st_size < 2**20:
    lines = 0
    with open(sample) as f:
        for _ in f:  # ітеруємо рядками    _  тут ім'я змінної, що обходить
            lines += 1
    print(lines)
```

```
121
```

Якщо рядок порожній, то він хибний. Якщо рядок містить тільки білі символи, то це можна визначити за допомогою методу `isspace` (методи класу `str`)

In [61]:
```
help(str.isspace)
```

```
Help on method descriptor isspace:

isspace(self, /) unbound builtins.str method
    Return True if the string is a whitespace string, False otherwise.

    A string is whitespace if all characters in the string are whitespace an
d there
    is at least one character in the string.
```

Модифікувати підрахунок кількості рядків файлу так, щоб додатково рахувалась кількість рядків файлу, що містять небілі символи (непорожні і не тільки з білих символів). У кінці вивести кількість рядків та кількість рядків з небілими символами.

In [62]:
```
if sample.stat().st_size < 2**20:
    lines = 0
    notempty_lines = 0
    with open(sample) as f:
        for s in f:  # ітеруємо рядками
            lines += 1
            if not s.isspace() and s is not None and s != "":# записати умов
                notempty_lines += 1
    print(lines, notempty_lines)
```

```
121 96
```

Написати функцію, що визначає для файла кількість його рядків з небілими символами.

```
def lines_counter(path):
    ???
    return ???
```

In [63]:
```
def lines_counter(path):
    notempty_lines = 0
    with open(path, encoding="utf-8", errors="ignore") as f:
        for s in f:
            if not s.isspace() and s is not None and s != "":
                notempty_lines += 1
    return notempty_lines
```

Перевіримо нашу фукцію зоча б трішечки.

In [64]:
```
lines_counter(sample)
```

Out[64]: 96

## Як зберегти табличну інформацію в csv-файл

Приклад даних. Табличні дані (як список контейнерів). (Використовувати списки чи кортежі тут різниці не має.) Та перелік заголовків до таблиці.

In [65]:
```
data = [(2025, 1, 13, 23456, 4.5, 99, 'текст1'),
        (2025, 7, 35, 23456, 4.5, 99, 'текст2'),
        (2025, 4, 34, 23456, 4.5, 99, 'текст3'),
        (2025, 11, 24, 23456, 4.5, 99, 'текст1')]
headers = ['h1', 'h2', 'h3', 'h4', 'h5', 'h6', 'h7']
```

Роботу виконуватиме бібліотека (документація), більш конкретно `writer` (документація) . Для читання є `reader` (документація).

In [66]:
```
import csv
```

In [67]:
```
with open('data.csv', 'w', encoding='utf-8', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(headers)
    for row in data:
        writer.writerow(row)
```

In [68]:
```
with open('data.csv', encoding='utf-8', newline='') as f:
    reader = csv.reader(f)
    headers = next(reader)
    lst = list()
    for row in reader:
        lst.append(row)
        print(row)
```

```
['2025', '1', '13', '23456', '4.5', '99', 'текст1']
['2025', '7', '35', '23456', '4.5', '99', 'текст2']
['2025', '4', '34', '23456', '4.5', '99', 'текст3']
['2025', '11', '24', '23456', '4.5', '99', 'текст1']
```

# Збирання задачі

Нехай протягом навчання програмні проєкти з різних предметів зберігалися в папках ... . Є перелік папок `folders`, в яких зберігаються ці проєкти (задає виконавець, виходячи з вмісту своїх носіїв даних).
*Примітка*. До цього переліку мають входити не окремі папки з проєктами, а папки, що містять, наприклад, всі проєкти такої-то дисципліни такого-то семестру тощо.

Інакше кажучи, не папка одного solution, а папка, в якій зберігаються solutions (якщо в термінології Visual Studio).

In [69]:
```
folders = ["/Users/yaroslavbyrak/PycharmProjects",
           "/Users/yaroslavbyrak/CLionProjects",
]   # задати перелік
folders
```

Out[69]:
```
['/Users/yaroslavbyrak/PycharmProjects', '/Users/yaroslavbyrak/CLionProject
s']
```

Щоб профільтрувати допоміжні файли середовищ, введемо "білий список" розширень ( `WHITE_LIST` ) та список розширень файлів з текстом програм(задається) ( `CODE_LIST` ).

Ці переліки були задані в умові.

In [70]:
```
for el in WHITE_LIST: print(el, end=' ')
```

```
.html .c .java .ipynb .json .h .hpp .cpp .txt .py .js .pyw .cs
```

In [71]:
```
for el in CODE_LIST: print(el, end=' ')
```

```
.c .java .h .hpp .cpp .py .js .pyw .cs
```

**Задача 1** Скільки різних розширень зустрічається у папках з заданого списку (тут і далі: включаючи вкладені), скільки файлів з кожним розширенням.
(Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)

Відповідну інформацію вивести на екран охайною табличкою. На рядку виводиться розширення, кількість файлів та ознака (C,W,O відповідно для файлів з текстом програм (відповідно до `CODE_LIST` ), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Наприклад,
```
.txt      155    W
.ipynb      73    W
.cpp      45    C
.exe       3    O
```

1. Створити порожній словник `d` . Його ключами будуть розширення, а значеннями відповідні кількості файлів.
2. Обійти всі папки з переліку `folders` . Для кожної папки запустити рекурсивний обхід, під час якого рахувати кількості файлів окремо для кожного

розширення.

3. Перетворити словник на список пар і відсортувати цей список по другій координаті в спадному порядку.
4. Обійти список пар і для кожного елемента вивести запитувану інформацію за допомогою f-рядка (налаштовуючи ширину поля виведення(, щоб досягти охайної таблиці.

Використати раніше розроблену функцію `kind`

In [72]:
```
d = {}
```

In [73]:
```
for i in folders:
    root = Path(i)
    s = root.glob("**/*")
    for el in s:
        if el.is_file() and el.suffix != "" and el.suffix is not None:
            if el.suffix not in d:
                d[el.suffix] = 1
            else:
                d[el.suffix] += 1

suffixes_cnts = list(d.items())
suffixes_cnts_sorted = sorted(suffixes_cnts, key=lambda el: el[1], reverse=T
```

In [74]:
```
for suffix, file_count in suffixes_cnts_sorted:
    print(f"{suffix:15} {file_count:10} {kind(suffix):10}")
```

```
.py                   7875 C
.pyc                  7002 O
.pyi                  3605 O
.json                 1530 W
.txt                  1273 W
.dat                  1084 O
.xml                   817 O
.js                    769 C
.cmake                 754 O
.o                     453 O
.cpp                   419 C
.bin                   366 O
.ninja                 363 O
.map                   246 O
.sample                210 O
.typed                 200 O
.c                     194 C
.iml                   193 O
.yaml                  189 O
.check_cache           186 O
```

```
.log                    182 O
.orig                    79 O
.woff                    75 O
.h                       67 C
.exe                     65 O
.out                     57 O
.j2                      32 O
.html                    29 W
.hpp                     26 C
.14                      24 O
.md                      22 O
.make                    20 O
.cfg                     18 O
.so                      18 O
.pxd                     16 O
.png                     15 O
.svg                     15 O
.css                     15 O
.pem                     10 O
.rst                     10 O
.rev                     10 O
.idx                     10 O
.pack                    10 O
.ps1                      9 O
.fish                     9 O
.csh                      9 O
.pth                      9 O
.eot                      9 O
.woff2                    9 O
.ttf                      9 O
.TAG                      8 O
.nu                       8 O
.sh                       8 O
.ico                      8 O
.yml                      7 O
.virtualenv               6 O
.dylib                    6 O
.pyx                      6 O
.lock                     6 O
.APACHE                   5 O
.BSD                      5 O
.lark                     5 O
.csv                      4 O
.gz                       4 O
.po                       4 O
.tab                      4 O
.marks                    4 O
.ts                       4 O
.toml                     3 O
.in                       3 O
```

```
.rsp                    3 0
.ipynb                  2 W
.1                      2 0
.pxi                    2 0
.crt                    2 0
.key                    2 0
.bz2                    2 0
.mo                     2 0
.bat                    2 0
.cmd                    2 0
.desktop                2 0
.tpl                    2 0
.tmpl                   2 0
.pot                    2 0
.jupyterlab-workspace         2 0
.d                      2 0
.zip                    1 0
.cff                    1 0
.unicode                1 0
.zi                     1 0
.SHA256SUM              1 0
.ixx                    1 0
.tx                     1 0
.internal               1 0
```

**Задача 2** За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний обсяг у байтах, кількість їх рядків (враховувати тільки для файлів категорії C; білі та порожні рядки не рахувати), середній розмір файлу, середня кількість рядків.

Відповідну інформацію вивести на екран охайною табличкою в хронологічному порядку.

Наприклад,

```
2024  09   23   123456  256   5368   11
2024  10   23   123456  256   5368   11
2024  11  134   123456  256    921   11
```

1. Створити порожній словник `d`. Його ключами будуть пари (рік, місяць), а значеннями відповідні кількості файлів, їх сумарний обсяг, кількість їх рядків.
2. Обійти всі папки з переліку `folders`. Для кожної папки запустити рекурсивний обхід, під час якого обробляти тільки файли з розширеннями з

білого списку. При обробленні файлу визначати рік та місяць його створення та оновлювати інформацію в словнику (за потреби додавати ключі).

3. Перетворити словник на список пар і відсортувати цей список за хронологією.
4. Обійти список пар і для кожного елемента вивести запитувану інформацію за допомогою f-рядка (налаштовуючи ширину поля виведення), щоб досягти охайної таблиці. Середні значення обчислювати під час обходу, не зберігаючи в структурі даних.

Використати раніше розроблені раніше функції `created_at`, `lines_counter`

In [75]:
```
YEAR = 2024
MONTH = 9
```

In [76]:
```
d = {}
```

In [77]:
```
for i in folders:
    root = Path(i)
    s = root.glob("**/*")
    for el in s:
        if not (el.is_file() and el.suffix is not None and el.suffix != "" a
            continue

        year, month = created_at(el)
        if year < YEAR or (year == YEAR and month < MONTH):
            continue

        line_count = 0
        size = el.stat().st_size
        if kind(el.suffix) == "C":
            line_count = lines_counter(el)

        if (year, month) not in d:
            d[(year, month)] = [1, size, line_count]
        else:
            d[(year, month)][0] += 1
            d[(year, month)][1] += size
            d[(year, month)][2] += line_count
```

In [78]:
```
d_items = list(d.items())
d_items.sort()
```

In [79]:
```
for key_time, file_info in d_items:
    year = key_time[0]
    month = key_time[1]
    file_count = file_info[0]
    size = file_info[1]
```

```
    line_count = file_info[2]
    avg_size = size // file_count
    avg_line_count = line_count // file_count
    print(
        f"{year:10} {month:8} {file_count:10} {size:15} {line_count:15} {avg
    )
```

```
      2024       10         43          393622            4480       9154
104
      2024       11        436         4219177           43694       9677
100
      2024       12        271         2611488           26885       9636
99
      2025        1         15          144557            1476       9637
98
      2025        2        118          733768           10586       6218
89
      2025        3        202         1429744           18461       7077
91
      2025        4        112          953987           11076       8517
98
      2025        5       1018        13808083          102547      13563
100
      2025        6        118         1024334           10743       8680
91
      2025        9        136         6522231           12451      47957
91
      2025       10        140         1269952           12939       9071
92
      2025       11        291         2336938           31611       8030
108
      2025       12         16          131481            1575       8217
98
      2026        2       3999        52508299         1234403      13130
308
      2026        3       5269       111543577         1882695      21169
357
```

**Задача 3** Для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, знайти таку інформацію: рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків (тільки для розширень категорії C; білі та порожні рядки не рахувати; для інших розширень цю клітину таблиці залишаємо незаповненою).

Цю інформацію записати в csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку додати рядок з заголовками: year, mon, ext, cnt, size, lines.

1. Створити порожній словник `d`. Його ключами будуть пари (рік, місяць,

розширення), а значеннями відповідні кількості файлів, їх сумарний обсяг, кількість їх рядків.

2. Обійти всі папки з переліку `folders`. Для кожної папки запустити рекурсивний обхід, під час якого обробляти тільки файли з розширеннями з білого списку. При обробленні файлу визначати рік та місяць його створення та оновлювати інформацію в словнику (за потреби додавати ключі).
3. Перетворити словник на список пар і відсортувати цей список за хронологією.
4. Зараз маємо список пар, де кожна компонента пари є послідовністю. З'єднати кожну пару в одну послідовність. Наприклад, пара ((2025, 3, '.cpp'), (1, 2, 4)) має бути замінена списком [2025, 3, '.cpp', 1, 2, 4]. Якщо останній елемент списку дорівнює нулю, замінити його значенням `None`. Тепер маємо список 6-елементних списків.
5. Вивантажити отриманий список 6-елементних списків у csv-file.

Використати раніше розроблені раніше функції `created_at`, `lines_counter`

*Примітка*. Не забороняється спочатку виконати задачу 3, а потім за її даними отримати результат задачі 2. Але це може виявитись зробити складніше, ніж у пропонованій вище схемі.

In [80]:
```
d = {}
headers = ["year", "mon", "ext", "cnt", "size", "lines"]
```

In [81]:
```
for folder in folders:
    root = Path(folder)
    for el in root.glob("**/*"):
        if not (el.is_file() and el.suffix in WHITE_LIST):
            continue

        year, month = created_at(el)
        ext = el.suffix
        size = el.stat().st_size

        lines = 0
        if ext in CODE_LIST:
            lines = lines_counter(el)

        key = (year, month, ext)

        if key not in d:
            d[key] = [1, size, lines]
        else:
            d[key][0] += 1
            d[key][1] += size
```

```
            d[key][2] += lines
```

In [82]:
```
d_items = list(d.items())
d_items.sort()
```

In [83]:
```
def create_csv(file_name, headers):
    path=""
    with open(file_name, 'w', encoding='utf-8', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(headers)
        path = f.name
    return path

def write_row(file_path, row):
    with open(file_path, "a", encoding="utf-8", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(row)
```

In [84]:
```
file_path = create_csv("lab1_1.csv", headers)
```

In [85]:
```
for key, value in d_items:
    row = list(key) + list(value)
    if row[-1] == 0:
        row[-1] = None
    write_row(file_path, row)
```